

# TaskFlow AI: A Goal-Driven Intelligent Task Scheduling Web Application

Sreelakshmi Gireesh

Master of Information Technology, University of Auckland  
Auckland, New Zealand | [sgir638@aucklanduni.ac.nz](mailto:sgir638@aucklanduni.ac.nz)

**Abstract** - TaskFlow AI is a web-based goal-driven intelligent scheduling assistant that helps users bridge the gap between long-term aspirations and daily task execution. Modern individuals frequently struggle to balance immediate responsibilities and meaningful long-term objectives due to manual planning tools that lack prioritisation and adaptive decision-making support. This often results in inefficiencies, reduced productivity, decision fatigue, and difficulty maintaining consistent progress toward important goals. Existing productivity applications are effective at storing tasks and events, but they generally provide limited assistance in determining what users should prioritise and when those activities should be performed. TaskFlow AI addresses these limitations by automatically decomposing user-defined goals into structured, actionable tasks and generating optimised daily schedules using a hybrid AI engine. The system dynamically organises tasks based on priority, deadlines, estimated effort, and user availability, enabling users to focus on execution rather than planning. By combining goal decomposition with intelligent scheduling, the application ensures that daily activities remain aligned with broader personal and professional objectives. In addition, TaskFlow AI incorporates adaptive planning mechanisms that allow schedules to be regenerated when priorities change or tasks are missed. Developed using Next.js 16, MongoDB, and a custom scheduling engine backed by a Large Language Model (LLM) layer, the application also features a gamified "Productivity Ocean" that visualises task-completion health as an evolving ecosystem. This report presents the system architecture, design rationale, implementation details, testing strategy, tooling decisions, and project management methodology. It further evaluates the effectiveness of the solution, discusses challenges encountered during development, and reflects on lessons learned throughout a three-sprint Agile development cycle.

**Keywords** - AI scheduling, goal management, LLM, gamification, MERN stack, task management, productivity, web application

## I. INTRODUCTION

Modern knowledge workers face a significant paradox: despite an abundance of digital productivity tools, consistent progress towards long-term goals remains elusive. Calendar applications and to-do lists excel at capturing commitments but transfer the entire cognitive burden of prioritisation and scheduling to the user. This systematically leads to *urgency bias* - reactive, short-term tasks crowd out important but non-urgent activities such as skill development, health maintenance, or personal

projects [1]. Research in behavioural economics confirms that humans are poor at self-scheduling under cognitive load; automated decision support significantly improves execution rates for planned tasks [2]. Existing AI-assisted schedulers such as Motion and Reclaim.ai automate time-blocking but derive schedules from flat task lists rather than structured goal hierarchies, leaving a meaningful gap between what users aspire to achieve and what they actually work on each day.

TaskFlow AI addresses these shortcomings by functioning as an active decision-support system rather than a passive scheduler. The system decomposes long-term goals into actionable tasks, then employs a hybrid AI engine to construct realistic daily schedules aligned with the user's stated priorities and available time windows. Dynamic re-planning adapts the schedule in real time when tasks are missed or new commitments arise. A gamified engagement layer sustains long-term motivation through XP rewards, level progression, and an ocean-health metaphor.

The three primary goals of the project are: (1) reduce decision fatigue by automating daily schedule construction from user-defined goals; (2) ensure all scheduled daily actions are explicitly and traceably linked to stated long-term objectives; and (3) increase task execution rates through motivational feedback mechanisms grounded in behavioural design principles.

The remainder of this report is structured as follows. Section II surveys related work. Section III describes the system design. Section IV details implementation. Section V covers testing. Section VI describes the project methodology. Section VII evaluates the tooling used. Section VIII discusses outcomes and limitations. Section IX concludes. Finally, the section X consists of the list of references.

## II. RELATED WORK

The design space for TaskFlow AI spans four overlapping product categories, each addressing part of the problem but none providing a holistic solution.

### A. Calendar Applications

Google Calendar and Apple Calendar represent the dominant paradigm for time management. They are excellent for scheduling fixed events and visualising commitments but rely entirely on manual user input. They offer no intelligence around what to schedule, how to

prioritise competing tasks, or how to adapt to changes during the day [3]. Users must perform all planning cognition themselves.

#### B. Task Management Tools

Notion and Todoist provide robust systems for capturing and organising work. Notion’s flexible block-based model suits complex knowledge work but demands significant manual configuration and provides no scheduling automation. Todoist simplifies task entry and supports priority flags but lacks the intelligence to place tasks on a calendar or to link tasks to higher-level goal structures [3].

#### C. AI-Based Scheduling Tools

Motion, Reclaim.ai, and Morgen represent the closest existing solutions. These tools automate time-blocking by analysing task lists and calendar availability. However, their goal models are shallow schedules that are generated from flat to-do lists rather than a structured goal hierarchy. There is no mechanism to ensure that the tasks being scheduled actually advance the user’s stated long-term objectives, nor do they provide meaningful feedback on goal progress over time [4].

#### D. Gamified Productivity Tools

Habitica and Forest demonstrate the motivational power of engagement mechanics. Habitica successfully uses XP, streaks, and rewards to drive habitual behaviour change. Forest promotes short-term focus sessions. However, both lack sophisticated scheduling logic and neither integrates goal decomposition. Duolingo’s streak and XP model is highly effective but domain-specific to language learning [65].

#### E. Goal-Oriented Timeline Tools

TimeStripe excels at decomposing long-term goals into milestones and daily actions, providing a clear temporal view from decade to day. Its limitation is the absence of any intelligent scheduling layer - users must still manually allocate time for each step. TaskFlow AI builds on this concept by adding an automated scheduling engine that places goal-derived tasks into the user’s calendar based on priority and availability.

The key differentiator for TaskFlow AI is the synthesis of three pillars: deep goal management, AI-driven adaptive scheduling, and gamification engagement. No single existing product combines all three into a cohesive, user-centric experience.

### III. DESIGN

#### A. System Architecture

TaskFlow AI is implemented as a monolithic Next.js 16 application following the App Router pattern. This architectural choice co-locates frontend and backend code, reducing the operational overhead of a separate Express server while providing a clear separation between React Server Components and API route

handlers. Figure 1 illustrates the high-level module dependency graph.

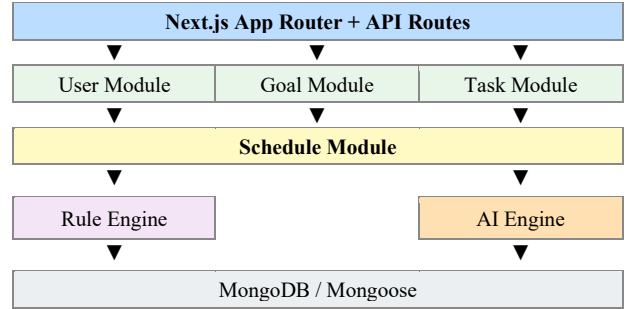


Fig. 1. High-level module dependency architecture.

The User Management module is the foundational dependency; all other modules reference users by ID. The Goal and Task modules manage the structured input to the scheduling layer. The Schedule Module consumes the self-contained Schedule Engine library, which is entirely decoupled from the HTTP layer to permit independent unit testing. MongoDB is accessed throughout via Mongoose 9 models.

#### B. Scheduling Engine Design

The scheduling engine is the technical centrepiece of the system. It exposes a unified interface accepting tasks, occupied blocks, a date, and an optional natural-language instruction, and returns a set of time-blocked schedule entries. Two interchangeable modes are supported, selected at runtime.

The **rule engine** sorts tasks by a composite key: priority rank (high=3, medium=2, low=1), then deadline urgency (ascending), then original insertion order. A cursor-based algorithm advances through the available scheduling window, skipping occupied blocks. This guarantees a complete, conflict-free schedule in  $O(n \log n)$  time, regardless of network or AI availability, and serves as a reliable fallback.

The **AI engine** submits a structured JSON prompt to an LLM containing serialised tasks, occupied blocks, and the optional instruction. The model responds with a task sequence, timing rationale, and a schedule summary. A post-processing validation layer clamps all output to legal time windows and falls back to the rule engine if the response is malformed or out-of-range. This hybrid design directly mitigates the identified risk of non-deterministic LLM outputs.

#### C. Feature Prioritisation

Features were prioritised using the MoSCoW method to effectively manage project scope and ensure the delivery of a functional minimum viable product (MVP) within the project’s three-sprint development timeline. This

prioritisation framework enabled the team to distinguish between essential functionality and desirable enhancements, ensuring that critical features were implemented first while reducing the risk of scope creep. By categorising requirements into Must-Have, Should-Have, Could-Have, and Nice-to-Have groups, the team was able to allocate development effort strategically, maintain focus on core project objectives, and adapt to time and technical constraints as they emerged throughout the development process. Table I summarises the resulting feature classification.

| Priority                  | Features                                                      |
|---------------------------|---------------------------------------------------------------|
| Must-Have                 | Auth, Task/Goal CRUD, AI Schedule, Calendar View, Re-planning |
| Should-Have               | Gamification XP, Streak system, Notifications                 |
| Could-Have                | Mood logging, Future-self simulation, Achievements            |
| Won't-Have / Nice-to-Have | Avatar customisation, Google Calendar sync                    |

TABLE I. MoSCoW Feature Prioritisation

#### D. User Interface Design

The user interface of TaskFlow AI is implemented using React 19 and Tailwind CSS 4, providing a modern, responsive, and highly interactive user experience. To ensure consistency across the application, a bespoke component library was developed within components/ui/foundation.tsx. This library contains reusable interface elements including buttons, panels, badges, alerts, modal components, and form controls, enabling a unified visual identity and reducing development complexity. The application adopts a dark-themed design language intended to minimise visual fatigue during extended usage sessions while maintaining a modern and professional appearance.

The system is organised into five primary views that collectively support the user journey from goal definition to task execution. As shown in Fig. 2, **Dashboard** provides an overview of user activity, displaying key productivity metrics, progress summaries, and upcoming scheduled work. The **Goals Management** view allows users to create, edit, and organise long-term and short-term goals, which serve as the foundation for subsequent task generation and scheduling. The **Tasks Management** interface enables users to manage actionable items associated with goals, including updating priorities, deadlines, and completion status.

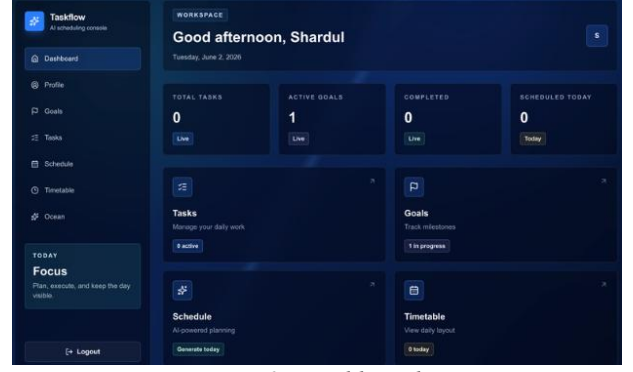


Fig. 2. Dashboard

The **Schedule/Timetable** view acts as the core execution interface of the application. It presents a chronological sequence of time-blocked activities for a selected day, allowing users to clearly visualise and follow their generated schedule. Each scheduled block can be individually marked as completed or missed using optimistic UI updates, providing immediate feedback without requiring a full-page refresh. This approach improves responsiveness and creates a smoother user experience. In addition, a “Regenerate Schedule” action allows users to dynamically adapt their plans when circumstances change. When activated, the current schedule state is submitted to the scheduling engine, which preserves completed activities while replacing remaining uncompleted tasks with a newly generated schedule based on updated priorities and constraints.

The final view, **Productivity Ocean**, introduces a gamified layer designed to improve engagement and motivation. User productivity is represented through a virtual ecosystem that evolves based on task completion and schedule adherence. As users consistently achieve their goals, the environment becomes healthier and more vibrant, providing visual reinforcement of positive habits and encouraging long-term commitment to productivity objectives.

#### E. Gamification: Productivity Ocean

The Productivity Ocean [Fig. 3] maps task-completion health to five metaphorical ocean states: empty, polluted, recovering, healthy, and thriving. XP is awarded per completed task, weighted by priority. An XP threshold table drives level progression, and a derived ocean-health score (0-100) determines whether sea-life sprites (dolphins, fish, seahorses) or rubbish sprites (bottles, cigarettes, plastic caps) are rendered. Achievement badges fire on milestone events such as the first task completion, a 7-day streak, or 100 tasks completed.

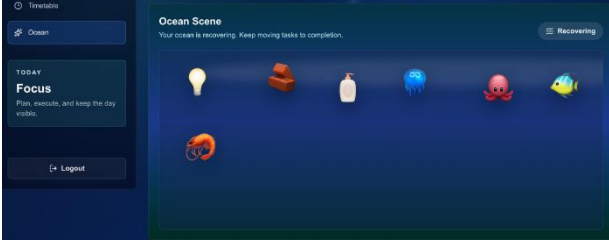


Fig. 3. Productivity Ocean

## IV. IMPLEMENTATION

### A. Backend API

The RESTful API is implemented as Next.js route handlers under `app/api/`. The authentication endpoints are: `POST /api/auth/signup`, `/login`, `/logout`, and `/me`. Goal management follows standard CRUD at `/api/goals` and `/api/goals/[id]`. Task management mirrors this at `/api/tasks`. Schedule endpoints include `GET/POST /api/schedules`, `POST /api/schedules/generate`, `POST /api/schedules/regenerate`, and a preview endpoint at `/api/schedules/engine` that accepts arbitrary task sets for testing. A free-time suggestions endpoint at `/api/free-time-suggestions` analyses the current schedule and recommends available windows.

### B. Schedule Engine Library

The engine library (`lib/scheduleEngine/`) contains four files. `types.ts` defines the shared TypeScript contract including `SchedulableTask`, `EngineOccupiedBlock`, `ScheduleEngineResult`, and `ScheduleGenerationMeta`. `ruleEngine.ts` implements the deterministic cursor-based algorithm. `aiEngine.ts` wraps the LLM call, handles JSON parsing, validates the response schema, and invokes the rule engine on failure. `mockEngine.ts` provides a deterministic stub for unit tests. The engine is completely stateless, accepting plain data structures and returning generated blocks with no database side effects.

### C. Authentication and Security

JWT tokens are issued on login and stored in HTTP-only cookies to mitigate XSS theft. Next.js middleware (`middleware.ts`) intercepts all requests to protected dashboard routes, extracts and verifies the token, and redirects unauthenticated users to `/login`. Passwords are hashed with `bcryptjs` before storage; the raw password is never persisted or logged. Token expiry and refresh logic ensures sessions are bounded.

### D. Productivity Ocean Module

The gamification module (`lib/productivityOcean.ts`) is entirely pure-functional. Helper functions—`calculateTotalXP`, `calculateLevel`, `getOceanStatus`, `flattenOceanItems`, and `calculateAchievements`—accept plain arrays of tasks and goals with no side effects, making the logic straightforward to test and reuse. The React client component (`components/productivity/ProductivityOceanClient.tsx`)

fetches task and goal data, computes ocean state client-side, and renders the animated sprite grid.

### E. Infrastructure

The application is fully containerised with a `Dockerfile` and `compose.yaml` in the repository root. Any team member can start the full stack application and MongoDB, with a single `docker compose up` command, eliminating environment drift across Windows, macOS, and Linux development machines. Production deployment targets Vercel (Next.js native), with MongoDB Atlas as the managed database.

## V. TESTING

### A. Unit Testing

Unit tests are co-located with the schedule engine library under `lib/scheduleEngine/__tests__/` and authored with the Vitest framework. Three test suites provide coverage: `ruleEngine.test.ts`, `aiEngine.test.ts`, and `mockEngine.test.ts`. The rule engine suite validates the primary sorting invariant: high-priority tasks must precede medium-priority tasks, which must precede low-priority tasks; ties in priority are broken by ascending deadline date; ties in deadline are broken by original insertion order. Additional tests assert that tasks whose estimated duration exceeds the remaining window are placed in the `unscheduled` list rather than overflowing the day, and that occupied blocks (completed or scheduled) are excluded from all candidate time slots. The AI engine suite uses a mock runner that returns pre-canned valid and invalid JSON responses. Valid-response tests verify that the post-processing layer correctly maps the LLM's task sequence onto concrete time blocks within the scheduling window. Invalid-response tests confirm that the fallback rule engine is invoked and produces a legal schedule when the LLM response is malformed.

### B. Integration and Manual Testing

Integration testing was conducted manually against the running development server. Test scenarios covered: account registration and login; creating, editing, and deleting goals and tasks; generating schedules in both rule and AI modes; marking individual blocks as completed or missed; triggering re-generation and verifying only non-completed blocks are replaced; and navigating the Productivity Ocean view while completing tasks to observe XP accrual, level progression, and achievement unlocking. API endpoints were tested using HTTP client tooling (curl and Postman), verifying correct status codes, error messages for malformed input, and authentication rejection for requests with missing or expired tokens.

## VI. METHODOLOGY

### A. Agile Development Process

The team adopted an Agile methodology with three one-week sprints, each targeting a clearly defined feature set. Sprint 1 covered project infrastructure setup, JWT authentication, and Goal/Task CRUD APIs with

Mongoose models. Sprint 2 delivered the scheduling engine, timetable UI, AI integration, and the engine preview endpoint. Sprint 3 focused on dynamic re-planning, the Productivity Ocean gamification module, bug fixing, and final polish.

Sprint planning sessions at the start of each week converted the MoSCoW backlog into specific Trello cards. Each card was assigned to an individual based on self-declared strengths across three tracks: frontend UI, backend API, and AI/algorithm. A daily asynchronous stand-up on Discord required each member to post blockers and progress, keeping the team aligned without synchronous overhead.

### B. Version Control Strategy

Version control followed GitHub Flow. The *main* branch was kept production-ready at all times. Every feature or fix was developed on a descriptively named branch (e.g., *feature/ai-schedule-gen*, *fix/rule-engine-overflow*). Completed branches were submitted as pull requests requiring at least one peer review before merging. This practice enforced shared code awareness, caught several integration incompatibilities before they reached *main*, and maintained a clean commit history for retrospective analysis.

### C. Scope Management

Scope creep was identified as the highest-likelihood, highest-impact risk. Mitigation was achieved through strict MoSCoW enforcement: Must-Have features were locked at the start of Sprint 1 and not renegotiated. New ideas raised during development were added to a backlog column on Trello for post-MVP consideration. This discipline ensured all Must-Have and the primary Should-Have feature (Productivity Ocean) were delivered on schedule.

## VII. TOOLING

**Next.js 16 / React 19.** The unified full-stack framework eliminated the need for a separate Express server, reducing operational complexity. App Router co-location of API and UI code improved developer experience. The main drawback was a non-trivial learning curve around React Server Components and streaming, which consumed Sprint 1 onboarding time.

**MongoDB / Mongoose 9.** The document model suited the evolving schema during early sprints, and Mongoose's validation hooks provided data integrity enforcement at the model layer. Connection pooling via a singleton *dbConnect()* helper prevented exhaust of serverless function database connections.

**Tailwind CSS 4.** Utility-first styling accelerated UI iteration significantly; layout changes required no context switching away from component files. The primary trade-off is verbose class lists that can obscure semantic intent;

the team partially mitigated this by abstracting common patterns into the foundation component library.

**Vitest.** The Jest-compatible API and native TypeScript support made Vitest straightforward to adopt without a separate compilation step. The absence of Vitest from the project's *package.json* scripts was a gap - tests were run manually during development rather than automatically on each commit, which allowed occasional regressions to go undetected until the next manual run.

**Docker / Docker Compose.** Full containerisation proved valuable: all six team members regardless of OS could reproduce the environment identically with one command. This prevented the 'works on my machine' class of integration failures entirely.

**GitHub Copilot and Claude (Generative AI).** AI coding assistants were used for boilerplate generation, TypeScript type scaffolding, and debugging assistance. Productivity on routine scaffolding tasks was noticeably higher. However, AI-generated code required careful domain-expert review: the assistant occasionally produced plausible but subtly incorrect type signatures and, in one instance, proposed a scheduling sort that violated the intended priority-then-deadline ordering. All algorithm-critical logic - the rule engine invariant and the AI engine fallback path - was written and reviewed manually without AI assistance. The team treated AI suggestions as a starting point rather than a final artefact.

## VIII. DISCUSSION

### A. Achievement of Goals

All three project goals were achieved. Decision fatigue was reduced by automating daily schedule generation from the user's goal and task inventory; users no longer need to manually sequence their day. Goal-to-task traceability was implemented via the *goalId* foreign key on every task, and the schedule engine propagates this linkage into generated blocks. Execution-rate motivation was addressed through the Productivity Ocean XP system and achievement badges.

All Must-Have features and the primary Should-Have (gamification) were delivered. Could-Have features - mood logging, future-self simulation, and the full achievement badge set - were correctly deferred to the backlog.

### B. Problems Encountered

The principal technical challenge was inconsistent LLM output formatting. Even with a strict JSON schema embedded in the prompt, the model intermittently returned responses with out-of-range time values or missing required fields. Implementing the validation and fallback layer resolved this in production but added approximately two days of unanticipated effort in Sprint 2.

A second challenge was Next.js App Router hydration mismatches between server and client renders of schedule data. These were resolved by moving schedule state management to client components with explicit loading states but required careful architectural refactoring mid-sprint.

The absence of a CI/CD pipeline was the most significant process gap. Without automated test runs on pull requests, two regressions in the rule engine sorting logic went undetected until manual testing in Sprint 3. Integrating Vitest into a GitHub Actions workflow is the highest-priority improvement for a future iteration.

## IX. CONCLUSION

TaskFlow AI demonstrates that AI-driven, goal-aligned scheduling is achievable within a constrained three-sprint student project timeline. The hybrid engine architecture, a deterministic rule base providing reliability with an LLM layer adding intelligence and natural-language flexibility, is the central technical contribution and is generalisable to any AI-assisted planning system where output correctness must be guaranteed.

Four lessons emerged from the project: (1) design and implement the fallback mechanism before the primary AI path; (2) decouple all business logic from HTTP handlers as early as possible to permit unit testing without a running server; (3) strict MoSCoW scope control is the single most impactful risk mitigation for a time-boxed project; and (4) generative AI coding assistants provide real productivity gains on scaffolding tasks but require careful domain-expert review on algorithm-critical paths.

Future work includes: integrating a CI pipeline with Vitest on every pull request; implementing user-facing streak tracking and push notifications; adding two-way Google Calendar synchronisation; introducing the mood and energy logging module to enable time-of-day task optimisation; and conducting formal usability testing with target users to validate the scheduling UX and inform the next design iteration.

## X. REFERENCES

- [1] Kahneman, D. (2011). *Thinking, fast and slow*. Psycnet.apa.org.
- [2] Google. (2024). *Google Calendar*. Accounts.google.com
- [3] 2026 Changelog. (2026). Todoist.com.
- [4] *Top 5 Best Shared Calendar Apps | Motion | Motion*. (2023). Usemotion.com.
- [5] Kirchner-Krath, J., Schmidt-Kraepelin, M., Schmähl, K., Christoph Schütz, Benedikt Morschheuser, & Sunyaev, A. (2025). Behavior Change Support Systems for Self-Treating Procrastination: Systematic Search in App Stores and Analysis of Motivational Design Archetypes. *Journal of Medical Internet Research*, 27, e65214–e65214.
- [6] Learning Agile. (2014). *Learning Agile*. O'Reilly | Safari.
- [7] OpenAI. (2023). GPT-4 Technical Report. *ArXiv:2303.08774 [Cs]*.
- [8] *DeepSeek Platform*. (2025). Deepseek.com.
- [9] Google. (2025). *Gemini*. Gemini.google.com; Google
- [10] Yuan, Y., & Yu, Y. (2025). Research on Intelligent Scheduling Algorithm and Learning Path Optimization of Physiology Curriculum Module in Vocational Colleges. *Proceedings of the 2025 3rd International Conference on Information Education and Artificial Intelligence*, 659–663.